UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
Facultad de Estudios Superiores Aragón

Reporte Primer Examen Parcial

Estructura de Datos



GRUPO: 1358

INTEGRANTES:

- Andalon Molina Victor Armando
- Hernandez Huerta Jossue Jassiel
- Carbajal Dominguez David
- Trujillo Quezada Pablo Mauricio
- Garcia Jorge Santiago Dazaev

13 de octubre de 2026

1. Qué es y qué hace

Es una lista de canciones que se puede recorrer hacia adelante y hacia atrás, y que es circular: después de la última canción vuelve la primera, y antes de la primera está la última. La lista siempre recuerda cuál es la **canción actual**, que es la que se usa para navegar y para simular la reproducción.

Con ella se puede:

- Agregar canciones al inicio o al final.
- Buscar una canción por su identificador, seleccionarla como actual o eliminarla.
- Eliminar la canción actual.
- Ver la canción actual, avanzar a la siguiente o retroceder a la anterior.
- Mostrar toda la lista en ambos sentidos y saber cuántas canciones hay.
- Simular la reproducción de *k* canciones, aunque *k* sea mayor que la cantidad de canciones.

Cada canción tiene un identificador único (se asigna solo, contando de uno en uno), título, artista y duración en segundos mayor que cero.

2. El nodo y los enlaces circulares

Cada canción vive dentro de un **nodo**. El nodo guarda la canción y dos enlaces: uno hacia el nodo anterior y otro hacia el siguiente. Los extremos también se enlazan entre sí, y así se forma el círculo.

```
[ 1 ] <-> [ 2 ] <-> [ 3 ] <-> [ 4 ]
  ^                            ^
inicio                       fin

El siguiente de [4] es [1].
El anterior de [1] es [4].
```

Si la lista está vacía no hay nodos. Si solo hay una canción, su nodo se enlaza consigo mismo en ambos sentidos.

3. Inicio, final y canción actual

La lista guarda tres referencias: dónde empieza, dónde termina y cuál es la canción actual, además de un contador.

- La **primera canción** que se agrega es inicio, final y actual a la vez.
- Agregar más canciones **no cambia la actual**; solo se mueve el inicio o el final.
- Si se elimina una canción que no es la actual, la actual se queda igual.
- Si se elimina la actual y quedan canciones, la nueva actual es la que venía después.
- Si se elimina la única canción, la lista queda vacía.

Antes de cambiar los enlaces se guardan los nodos vecinos, para no perder ninguno. Al eliminar un nodo, nada de la lista queda apuntándole; Java lo libera solo cuando ya no se puede alcanzar.

4. Qué tan rápida es cada operación

n es la cantidad de canciones.

Operación	Tiempo	Por qué
Agregar al inicio o al final	O(1)	Se conocen los extremos; no hay que recorrer nada.
Ver la canción actual	O(1)	Ya se tiene la referencia.
Avanzar o retroceder	O(1)	Solo se pasa al nodo vecino.
Eliminar la canción actual	O(1)	El nodo ya está localizado; solo se reenlazan sus vecinos.
Saber cuántas canciones hay	O(1)	Se lee el contador.
Buscar o seleccionar por identificador	O(n)	Puede haber que revisar todas.
Eliminar por identificador	O(n)	Primero se busca (O(n)); después se quita (O(1)).
Mostrar todas las canciones	O(n)	Se visita cada una una sola vez.
Simular k reproducciones	O(k)	Se avanza k veces.

5. Comparación con un arreglo

	Arreglo	Esta lista
Acceso	Por posición, muy rápido.	Rápido a la actual y a los extremos; por posición hay que recorrer.
Inserción	Al inicio hay que mover todos los elementos; si se llena hay que copiarlo.	Al inicio y al final es inmediato, sin mover nada.
Eliminación	Hay que mover los elementos para tapar el hueco.	Inmediata si ya se tiene el nodo (la actual).
Memoria	Un bloque continuo, pero con espacio reservado sin usar.	Solo lo necesario, pero cada canción lleva dos enlaces extra.

Para una lista de reproducción, donde se avanza, se retrocede y se agregan o quitan canciones en los extremos, la lista enlazada es más cómoda y eficiente.

6. Pruebas y problemas encontrados

Se hicieron 23 pruebas y todas salieron bien. Cada una indica el estado inicial, la operación, el resultado esperado y el obtenido, y después de cada cambio se revisa la cantidad de canciones y que los enlaces sigan correctos. Cubren: lista vacía, una sola canción, inserciones en ambos extremos, recorridos con cinco canciones, eliminar primera, última, intermedia, actual y única, identificadores que no existen, simulación con más de una vuelta y entradas inválidas.

Problemas que había que evitar:

- **Ciclos infinitos:** en una lista circular nunca aparece *null*, así que los recorridos se controlan con el contador.
- **Actual apuntando a una canción eliminada:** se mueve a la siguiente, o a null si la lista queda vacía.
- **Perder nodos al cambiar enlaces:** se guardan los vecinos antes de modificar.
- **Datos inválidos:** una canción con datos incorrectos se rechaza sin gastar un identificador.

7. Contribución de cada integrante

Andalon Molina Victor Armando: Programó las clases Cancion y Nodo, con la validación de los datos de la canción (título, artista y duración mayor que cero).

Hernandez Huerta Jossue Jassiel: Programó la inserción al inicio y al final, el enlace circular entre los extremos y la navegación (avanzar y retroceder).

Carbajal Dominguez David: Programó la búsqueda y selección por identificador, la eliminación por identificador y de la canción actual, y el manejo de la actual al eliminar.

Trujillo Quezada Pablo Mauricio: Programó el menú, la lectura y validación de entradas, la muestra de la lista en ambos sentidos y la reproducción simulada.

Garcia Jorge Santiago Dazaev: Diseñó y ejecutó las pruebas, redactó el README y el reporte técnico y preparó el repositorio de GitHub.